Explaining the Behaviour of Game Agents Using Differential Comparison

Ezequiel Castellano
ecastellano@nii.ac.jp
National Institute of Informatics
Tokyo, Japan

Xiao-Yi Zhang
xiaoyi@nii.ac.jp
National Institute of Informatics
Tokyo, Japan

Paolo Arcaini
arcaini@nii.ac.jp
National Institute of Informatics
Tokyo, Japan

Toru Takisaka
takisaka@uestc.edu.cn
University of Electronic Science and Technology of China
Chengdu, China

Fuyuki Ishikawa
f-ishikawa@nii.ac.jp
National Institute of Informatics
Tokyo, Japan

Nozomu Ikehata
Konami Digital Entertainment Co., Ltd.
Tokyo, Japan

Kosuke Iwakura
Konami Digital Entertainment Co., Ltd.
Tokyo, Japan

ABSTRACT

The difficulty in exploring the game balance has been increasing, especially in Game-as-a-Service (GaaS) with updates in every few weeks, and due to the complexity in game design and business models. In the limited time available for testing, using automated game agents enables much more test plays than using human test players does, and it has been accelerated by the recent progress of deep reinforcement learning. However, understanding specific behaviours of each agent is hard due to their "black-box" nature. In this paper, we propose a method for explaining the behaviour of game agents using differential comparison between agents. This comparison approach is motivated by our experience with existing explanation techniques that often extracted uninteresting, common aspects of the behaviour. In addition, there are large potentials for the application of the comparison: between agents with different learning algorithms, between human agents and automated agents, and between test agents and users. We applied our technique to a prototype of a commercial GaaS and confirmed our technique can extract specific differences between agents.

CCS CONCEPTS

• **Theory of computation → Reinforcement learning**; • **Software and its engineering → Software testing and debugging**.

KEYWORDS

reinforcement learning, games, testing, explainability

ASE '22, October 10–14, 2022, Rochester, MI, USA

ACM ISBN 978-1-4503-9475-8/22/10…$15.00
https://doi.org/10.1145/3551349.3560503

ACM Reference Format:
Ezequiel Castellano, Xiao-Yi Zhang, Paolo Arcaini, Toru Takisaka, Fuyuki Ishikawa, Nozomu Ikehata, and Kosuke Iwakura. 2022. Explaining the Behaviour of Game Agents Using Differential Comparison. In *37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22), October 10–14, 2022, Rochester, MI, USA.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3551349.3560503

1 INTRODUCTION

Game-as-a-Service (GaaS) [21] is a popular type of game which is continuously updated (every few weeks) with new contents to keep the players engaged. Different online simulation games (that simulate activities of the real world) are based on this paradigm. In this setting, new updates can bring new features that can be used by the virtual characters (e.g., new weapons), new tasks to complete (e.g., fighting new enemies), etc. A game of this type has been provided by the industrial partner and motivates our work.

For these games, the *game balance* is an important aspect that must be guaranteed to make a game successful. Game balance must guarantee a proper trade-off between "pay to win" and "play more to win", i.e., a right balance between players buying the new introduced features and those playing longer to achieve them; different conflicting aspects should be considered for this trade-off: (i) the players should desire to keep on playing in the long term; (ii) some players should be willing to pay to obtain some features; (iii) the number of players playing the game (also those who are not willing to pay) should be maximized; (iv) the revenue of the company should be maximized in the long term, i.e., the interest in the game by the users should remain high over time.

For the game company, it is very important to assess (i.e., *test*) the new introduced features before releasing them, to check if they provide a good game balance. To do this, several test plays must be played. However, using human players as testers is not feasible not only because it is too costly, but also because the short-release time (few weeks) makes the time available for testing limited (note that the game played by a human can take up to hours). To tackle

these issues, *game agents* have been used to test new game releases in GaaS; typical game agents are based on reinforcement learning techniques [2, 24], or also rule-based ones. The usage of game agents allows for automatizing and speeding up the testing process. Moreover, it allows for simulating different types of agents, more or less skilled ones, and with different levels of willingness to pay.

However, a fundamental step for the analysis of the test executions is to understand "why" an agent behaves in a given way. For example, if an agent wins too easily, the game company would like to understand what the agent is considering in its decisions, as this type of reasoning could also be applied by a human player, so breaking the game balance. Understanding the behaviour of a game player is far from trivial, in particular those based on AI techniques, like deep reinforcement learning.

Contribution. In this paper, motivated by our collaboration with the game company, we tackle the problem of explaining game agents. When starting working on this problem, we soon realized that explaining the behaviour of an agent alone is difficult, as there is no reference to make an explanation, and, when possible, not interesting; indeed, common techniques that allow to explain a single agent lead to very simple and apparent explanations. On the other hand, explaining the "difference" between two agents is easier, as it allows for understanding which are the features of the game state that are considered by a specific agent when applying an action. Moreover, it is also more interesting, as it opens different opportunities, such as comparing agents with different abilities, that is important to assess game balance.

So, we propose an approach, inspired by differential testing [6], to compare game agents. Given two agents under comparison, we first execute them starting from several possible states of the game and we record which actions the two agents apply. Then, we check which are the values of the state variables when the two agents apply the same action and when they apply different actions. Finally, by using suitable statistical tests, we provide an "explanation" of the agents behaviours in these two aspects:

(1) Explanation of *agreement*: a variable x could be the reason of agreement if its value is statistically significantly higher/lower when the two agents agree on a specific action a_i than when the second agent applies a different action a_j;
(2) Explanation of *disagreement*: a variable x could be the reason of disagreement if its value x is statistically significantly higher/lower when the two agents apply two different actions a_i and a_j, than when they apply the same action a_i.

Paper structure. Section 2 introduces the problem definition in more details. Section 3 provides the minimal definitions needed. Then, Sections 4 and 5 introduce the proposed approach, and present examples of the results provided as output. Section 6 discusses some threats that may affect the validity of the approach, and Section 7 overviews some related works. Finally, Section 8 concludes the paper.

Remark 1. For IP protection, we cannot mention the specific game targeted in this work nor provide specific details on it. For the same reason, we cannot show complete experimental results nor provide explicit names of the described actions and state variables. Still, we will try to give the intuition of the type of game we target and of the type of insights that can be obtained with our approach.

2 PROBLEM STATEMENT

We here first introduce the problem of game balance in Section 2.1, and how game agents are used to test it in Section 2.2. Then, we explain the difficulty of explaining the behaviour of game agents in Section 2.3, and motivate our explanation approach in Section 2.4.

2.1 Game Balance Problem in GaaS

The presented work is motivated by a specific game provided in the form of Game-as-a-Service (GaaS) [21]. It is frequently updated in every few weeks, not in the form of one-shot release or batch update over months.

The most significant aspect in such GaaS settings is the game balance of each periodical update. Specifically, a good game balance should motivate users play the new update to obtain benefits inside the game such as character enhancement at a previously-unreachable level. On the other hand, this incentive should not be too much for various reasons. Too large character enhancement, or such, let users lose motivations as what they obtained with their previous plays is invalidated. It also leads to "inflation" where the game operator needs to continuously make radical changes and invent new mechanisms to make difference. The game balance also matters on the "pay to win" and "play more to win" aspects. Paying more or playing more should allow for benefits inside the game but too much difference will let lightweight users give up.

The problem of game balance has existed for traditional game products and is listed as one of the key differences from common software testing [17]. It is much more difficult in GaaS due to the short period of development cycles as well as the complex game design with the "pay to win" and "play more to win" aspects.

For this problem of game balance, it is essential to have intensive test plays with goals in quantitative metrics, e.g., "10 hours standard play should allow to obtain the new item with more than 98% probability" or "users should not reach levels more than 80". Results of test plays can be evaluated with such quantitative goals.

2.2 Game Agents for Automated Test Plays

Traditionally, test plays have been conducted by human test players. However, the speed and amount of test plays are limited, especially for GaaS with very frequent updates.

In the case of the target game, one play requires several tens of command inputs and thus 30 minutes at least even without deep thinking. Moreover, there are a variety of configurations we want to test such as characters, play strategies, payment levels, and so on.

To allow for large amount of tests in the limited time, the use of automated game agents has been actively investigated. The recent technical advance has promoted use of reinforcement learning, especially based on deep neural networks, for building such game agents [2, 24]. We have also explored other approaches such as rule-based agents in which parameters are optimized through tests, e.g., thresholds to choose some action. As expected, game agents provided clear benefits by accumulating much more data and statistics for the target game.

2.3 Difficulty in Explaining Behaviour of Game Agents

Through our investigation of game agents for testing the game balance, we found that a critical problem is that it provides limited insights in terms of the play strategy. For example, suppose values of the goal metric achieved by the game agents were higher than expected. This might be because of the configuration parameters of the game, i.e., too easy, but another possibility is that the game agents found unknown good play strategies. In any case, it is essential to intuitively understand how the agents played and succeeded (or failed). Manual investigation of the play logs for several tens of steps is too costly and contradicts with the original motivation for efficient testing.

Explainability or interpretability has been considered as a critical problem of general Artificial Intelligence systems, especially those with deep neural networks. Given the wide acceptance of supervised learning techniques, explanation techniques for classification models have been investigated intensively [4]. Explanation techniques for reinforcement learning are also emerging as surveyed by Puiutta et al [16].

In the survey study, explanation techniques for reinforcement learning are classified in terms of when the explanation is extracted and the target scope of explanation. Our interest is in the model-agnostic (thus post-hoc) and global explanation. We do not want to limit the agent models to explore the possible maximum performance. We are now interested in explanation of the "global behaviour" of agents (i.e., when, in general, they play a given action), though, in the future, we may want to have local explanation for one specific action in a specific state.

Techniques for model-agnostic, post-hoc, and global explanation try to extract simpler models such as decision trees from test logs [5, 9]. However, our trials revealed such techniques are difficult to employ. Models such as decision trees approximating the complex behaviour are still too large and complex for human to intuitively understand. We could put some measures to limit the complexity of the extracted models, e.g., extract simple rules rather than decision trees. Still, the most critical point is that the explanation is dominated by uninteresting, common behaviour in the target game, e.g., "use a recovery item when the energy is too low".

2.4 Motivation and Approach

With our accumulated experience with the target game, we realized that our implicit requirement is to explain *unique or specific behaviour of each game agent*. This was difficult with existing explanation techniques for one agent and we therefore investigate *explanation via differential comparison between two agents*. As motivated above, this approach will allow to extract non-common behaviour of each agent, which can be then analyzed over their performance. There are also many potential targets of comparison as follows.

- Comparison between different game elements such as characters and items, sometimes relevant with the payment levels and length of the play.
- Comparison between agents with different training algorithms, e.g., A3C and PPO for reinforcement learning.
- Comparison between test plays before release and user plays in operation.
- Comparison between automated agents and human players.

In the next sections, we present the proposed approach. We first introduce basic definitions in Section 3. We then introduce the generation phase of the approach in Section 4, and the analysis phase in Section 5.

3 DEFINITIONS

Here we give definitions necessary to describe our concrete techniques in the next sections. Our work is motivated by the experience in a specific game, but can be generalized over games having the same structure of state-action pairs.

A game player is called *agent* in our context. Normally, the agent is the human user of the game, but it can also be an *intelligent agent* trained or programmed to play the game (e.g., for testing purposes). This is the setting in which operate, where we want to assess the behaviour of different intelligent agents for testing purposes.

Definition 1 (Game state)**.** The *state* s of a game is given by a set of *variables* $\overline{x}$ that describe the status of the simulated characters and of the virtual world. We will identify with $\mathcal{S}$ the set of possible states.

Examples of variables are the stamina of the characters, items available to the characters (e.g., weapons, medicine), events that are happening in the virtual world (e.g., the enemies are approaching), etc.

Definition 2 (Action)**.** An *action* a is an operation that can be applied by the agent to affect the current state of the game. We identify with $\mathcal{A}$ the set of possible actions. Given a state s, there is a set of available actions $avActions(s) \subseteq \mathcal{A}$ that can be *applied* (or *played*) by the agent in the state.

Possible actions are shooting a certain enemy, taking a medicine, moving to a different place, etc.

Definition 3 (Simulation)**.** Given a state s and an action a, the next state s' is given by the application of the action to the state. Function $\texttt{step}\colon (\mathcal{S} \times \mathcal{A}) \rightarrow 2^{\mathcal{S}}$ defines the transition function; if $\texttt{step}(s, a) = \emptyset$, it means that the action a is not an available action in state s.
A *simulation* is a sequence of states $(s_1, a_1), \ldots, (s_{n-1}, a_{n-1}), (s_n, -)$, such that $s_{i+1} \in \texttt{step}(s_i, a_i)$ for $i \in \{1, \ldots, n-1\}$.

Definition 4 (Agent and agent policy)**.** An *agent* $\texttt{AG}_X$ is either a human player or a software (AI-based, rule-based) that plays the game. The agent defines a *policy* to choose actions, i.e., $\texttt{AG}_X(s) = a$ with $a \in avActions(s)$ for each $s \in S$.[1]

The policy implemented by an agent could be very difficult to understand, in particular for AI-based agents trained using deep reinforcement learning. In this paper, we aim to *explain* the policy of an agent. In order to do this, we need to create tests exposing the agent's decisions. A test is defined as follows.

[1]Note that, in this work, we assume that the agent's policy is deterministic, because this is the case for the agents that we test in this work. However, in general, the agent's policy could be non-deterministic.

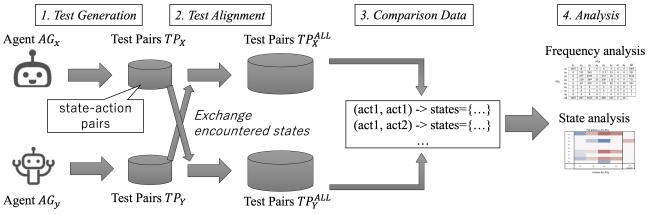

Figure 1: Overall differential comparison approach

Definition 5 (Test and test pair)**.** A *test* is given by a state s, meaning that the agent $\mathtt{AG}_X$ under test must select an action from $avActions(s)$. The *test result* is the chosen action, i.e., $\mathtt{AG}_X(s) = a$. We call (s, a) as a *test pair*.

Note that a test only considers the current state in which an action is played. However, an agent's policy could also consider the history of previous states to decide the next action. We leave as future work the use of this more complex type of tests that consider multiple previous states.

4 PROPOSED APPROACH - GENERATION OF COMPARISON DATA

In this paper, we aim at explaining the behaviour of a game agent. In particular, as explained in Section 2.4, we want to explain the behaviour of one agent w.r.t. the behaviour of another agent.

Our process is shown in Fig. 1 and explained in this section and in the following one. Intuitively, we collect actions of each agent for the same set of states (steps 1 and 2 in the figure). Then, we build a data structure (step 3 in the figure) that records the states in which the two agents took the same action (i.e., they *agree*) and the states in which they took different actions (i.e., they *disagree*). We can run different types of statistical analysis over the obtained data structure (step 4 in the figure).

Our method provides the necessary first step for the assessment of the game balance. The next step of the assessment would be to use quantitative metrics (e.g., obtained score, time spent in playing, money spent) to evaluate each test or a set of tests; then, using the comparison results provided as output by our approach, we could try to find correlations between the obtained metrics values and the performed actions. Note that this paper focuses on the agent comparison only; the final steps of the assessment of game balance are part of our future work.

We applied our method to a GaaS in operation and compared a reinforcement learning agent and a rule-based agent as well as reinforcement learning agents with different algorithms.

4.1 Test generation

The first step of the approach consists in generating different simulations for each agent. Given an agent $\mathtt{AG}_X$, we generate *NS* simulations of *n* states, so obtaining a set of simulations as follows:

$$Sims_X = \{[(s_1, a_1), \ldots, (s_{n-1}, a_{n-1}), (s_n, -)]_{j=1,\ldots,NS}\}$$

From $Sims_X$, we extract the set of test pairs as follows:

$$TP_X = \bigcup_{sim \in Sims_X} \bigcup_{(s,a) \in sim} \{(s, a)\} \tag{1}$$

4.2 Test alignment

Since we want to compare the behaviour of two agents $\mathtt{AG}_X$ and $\mathtt{AG}_Y$, we need to observe their actions from the same states. So, given the tests of one agent $\mathtt{AG}_X$, we need to observe the behaviour of $\mathtt{AG}_Y$ on the same tests; concretely:

$$TP_Y^X = \bigcup_{(s,a) \in TP_X} \{(s, a_Y)\} \quad \text{with } a_Y = \mathtt{AG}_Y(s) \tag{2}$$

So, the total set of test pairs for one agent is given by those coming from their own simulations (Eq. 1) and from those obtained by executing the tests of the other agent (Eq. 2), i.e.,

$$TP_X^{all} = TP_X \cup TP_X^Y$$

$$TP_Y^{all} = TP_Y \cup TP_Y^X$$

Thus, actions are collected for all of the states that were encountered by either of the two agents.

Remark 2. Note that doing test alignment requires to be able to initialize the game from any state. For some games, this could be not trivial. For the target game, this cannot be done directly, but required us to "drive" the simulation to that state.

4.3 Comparison data for agents' policies

In this phase of the process, we want to observe when (i.e., in which states) the two agents apply the same actions, meaning that their policies *agree*, and when they apply different actions, meaning that their policies *disagree*. To do this, for each pair of actions $(a_i, a_j) \in \mathcal{A} \times \mathcal{A}$, we collect the states in which agent $\mathtt{AG}_X$ decides to apply a_i and $\mathtt{AG}_Y$ decides to apply a_j, i.e.,

$$CompStates(a_i, a_j) = \left\{ s \in \mathcal{S} \colon (s, a_i) \in TP_X^{all} \wedge (s, a_j) \in TP_Y^{all} \right\} \tag{3}$$

Intuitively, $CompStates(a_i, a_i)$ (with $i \in \{1, \ldots, |\mathcal{A}|\}$) are the states in which agents agree in doing action a_i, while states $CompStates(a_i, a_i)$ (with $i \neq j$) are those in which they disagree and $\mathtt{AG}_X$ chooses a_i while $\mathtt{AG}_Y$ chooses a_j.

5 PROPOSED APPROACH - ANALYSIS PHASE

Starting from the sets of states $CompStates(a_i, a_j)$ for all the pairs of actions, we can perform different types of analyses to understand similarities and differences between the two agents' policies.

5.1 Frequency analysis

In the first type of analysis, we perform descriptive statistics to understand how often the two agents agree and disagree. Specifically, we build a *frequency matrix* as shown in Fig. 2, where we report the number of states in each $CompStates(a_i, a_j)$. Intuitively, this type of analysis gives a high level understanding of how often the two agents agree and disagree for which types of actions.

5.1.1 Possible insights obtained from the results. Fig. 2 reports an example of obtained frequency matrix. Note that the action names have been anonymised. In the following, we illustrate which types of insights can be obtained from the frequency matrix, and also explain some of the insights we got for our specific targeted game:

AG_Y \ AG_X	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	all
a_1	1067	0	0	0	0	0	0	0	1067
a_2	0	121	263	1	113	33	0	0	531
a_3	0	277	2953	9	357	18	0	0	3614
a_4	0	120	209	21	250	111	0	0	711
a_5	0	89	233	0	159	20	0	20	521
a_6	0	0	0	0	0	0	0	0	0
a_7	0	0	2	0	0	0	0	0	2
a_8	0	0	0	0	1	0	0	0	1
all	1067	607	3660	31	880	182	0	20	

Figure 2: Frequency matrix ($|CompStates(a_i, a_j)|$)

- *Complete agreement.* For some actions like a_1, the two agents always agree in applying it, i.e., it never happens that one agent decides to apply it while the other agent selects a different action. It could be that the choice of the action is trivial for any reasonable agent, or that both agents implement a very similar reasoning in the situations that lead to the selection of that action. In our target game, a_1 is an action that is somehow imposed by the game, and so it is played by both agents when required.
- *Strong disagreement.* For some specific action, the disagreement of one agent with the other is significant and spread across different actions. For example, AG_Y widely disagrees with AG_X on action a_4; when agent AG_Y plays a_4, agent AG_X often plays different types of actions (four of them) and plays the same action only $21/711$ of the times. This means that, while AG_X considers different aspects leading to different decisions in the different states, AG_Y consistently considers action a_4 the best choice; in the target game, a_4 is a very conservative action that allows the character to save energy that can be spent later in the game, but that does not allow to improve the score in the short term. This shows that AG_Y is a more precautious agent that does not take too much risks, while AG_X is more willing to take risky actions.
- *Mild disagreement.* For some other actions like a_3, instead, although there is some disagreement in some states, overall the two agents agree in the large majority of the states; a_3 is played in 2953 states by both agents, out of the 3660 states in which it is played by agent AG_X and out of the 3614 states in which it is played by agent AG_Y. In our target game, a_3 is the main action that makes the game evolve, and so it makes sense that it is often played by both agents.
- *Action ignored by one agent.* Some actions are never played by one agent, but played frequently by the other agent. This is the case of action a_6 that is never played by AG_Y but it is played quite often by AG_X. This could be due to different reasons:
  - Action a_6 could be a risky action that can provide big benefits but also great losses. Agent AG_Y is a precautious agent that does not accept such level of risk that, instead, is accepted by agent AG_X. This is the explanation for the target game in which a_6 can lead to some risk for the character in some cases; AG_Y never prefers to play it, as it is a very precautious player. Note that the precautious nature of AG_Y was already witnessed by the observations for a_4 in the *strong disagreement* category.
  - In a different setting (i.e., not for the two agents considered in our results), it could be that AG_Y is not even aware of the possibility of playing a_6. This could be the case in which AG_Y is an agent trained for an old version of the game, and a_6 has been introduced in the new release for which, instead, AG_X has been trained. Note that also this comparison among agents trained for different versions of the game is very interesting for the game company; indeed, this analysis can show whether the newly introduced feature (i.e., the new available action) leads to a change in the playing strategies of the players or not.
- *Rare actions.* Some actions are never played by one agent and seldom played by the other agent. This is the case of action a_7 that is never played by agent AG_X and played only twice by AG_Y. a_7 is an action that has not a big impact (neither positive nor negative) on the progression of the game; the agent can play it for it own amusement, but this does affect the continuation of the game. This is why the action is so seldom played. This type of action is not too interesting, and can be safely ignored for the comparison of the agents.

5.2 State analysis

The frequency analysis presented in Section 5.1 allows to understand how often two agents agree and disagree, but not the "reasons" for the agreement and disagreement. In this section, we propose a type of analysis to do this; given an action a_i of agent AG_X, we want to *characterize* the states in which AG_Y plays the same action and the states in which AG_Y plays a different action. To do this, we check the values of the state variables. Specifically, given a variable $x \in \overline{x}$, and the set of states $CompStates(a_i, a_j)$ (see Eq. 3), we collect the values of x across the states, i.e.,

$$values^{x}_{a_i,a_j} = \bigcup_{s \in CompStates(a_i,a_j)} \{[\![x]\!]_s\}$$

where $[\![x]\!]_s$ is the value of variable x in state s.

Starting from these distributions of values, we want to understand which are the features of the states that contribute to the agreement and disagreement of two agents AG_X and AG_Y in the selection of the action.

5.2.1 Disagreement check. To check the disagreement, we proceed as follows. Given actions a_i and a_j (with $i \neq j$) played by agents AG_X and AG_Y, we want to understand why AG_Y played a different action.[2] We do this by checking if the value of a variable $x \in \overline{x}$ was a possible cause of the disagreement; namely, we check if the value of x is significantly different (and how (greater or lower) and how much different) from the states in which both agents apply action a_i:

- we compare distributions $values^{x}_{a_i,a_j}$ and $values^{x}_{a_i,a_i}$ with the Mann–Whitney U test which is a nonparametric test telling whether two distributions are significantly different[3]; if the

[2] Note that the analysis is not commutative, i.e., a similar analysis must be done to understand why AG_X did not play the action of AG_Y.

[3] Note that we use a nonparametric test as the compared distributions are, in general, not normal; we have used the Shapiro-Wilk test to check normality of the distributions.

p-value is less than the significance level $\alpha = 0.05$, we can reject the null hypothesis that there is no difference between the two distributions. If this is the case, it means that the value of x is a possible factor of the disagreement;
- if there is a significant difference, we check the *direction* and the *strength* of the significance with the $\hat{A}_{12}$ effect size:
  - If $\hat{A}_{12}$ is greater than 0.5, it means that the value of x is usually higher when the agents play the two different actions a_i and a_j than when they play the same action a_i; the strength of the significance can be categorized as follows [10]: *negligible* when $\hat{A}_{12} \in (0.5, 0.556)$, *small* when $\hat{A}_{12} \in [0.556, 0.638)$, *medium* when $\hat{A}_{12} \in [0.638, 0.714)$, and *large* when $\hat{A}_{12} \geq 0.714$.
  - Similarly, $\hat{A}_{12}$ lower than 0.5 means that the value of x is usually lower when the agents play the two different actions a_i and a_j than when they play the same action a_i; the strength categories are: *negligible* when $\hat{A}_{12} \in (0.494, 0.5)$, *small* when $\hat{A}_{12} \in (0.362, 0.494]$, *medium* when $\hat{A}_{12} \in (0.286, 0.362]$, and *large* when $\hat{A}_{12} \leq 0.286$.

Number of checks. Disagreement checks are done for each pair of actions a_i, a_j (with $i \in \{1, \ldots, |\mathcal{A}|\}$, $j \in \{1, \ldots, |\mathcal{A}|\} \setminus \{i\}$) for which it holds $|CompStates(a_i, a_j)| \geq 5$ and $|CompStates(a_i, a_i)| \geq 5$; this is required for the application of the statistical test. Fig. 2 shows that, in our case, disagreement checks between 15 pairs of actions are done; for each pair of actions that can be compared, disagreement checks are done for all the state variables.

5.2.2 Agreement check. To check the agreement among the two agents $\mathtt{AG}_X$ and $\mathtt{AG}_Y$ in the selection of an action a_i, we proceed in a similar way. We first collect all the states in which $\mathtt{AG}_X$ selects action a_i and $\mathtt{AG}_Y$ selects a different action a_j (with $j \neq i$):

$$values^x_{a_i, a_{j \neq i}} = \bigcup_{j \in \{1,\ldots,|\mathcal{A}|\} \setminus \{i\}} values^x_{a_i, a_j}$$

Then, we compare distributions $values^x_{a_i,a_i}$ and $values^x_{a_i,a_{j\neq i}}$ as explained before for the disagreement case.

Number of checks. Agreement checks are done for each action a_i (with $i \in \{1, \ldots, |\mathcal{A}|\}$) for which it holds $|CompStates(a_i, a_i)| \geq 5$ and $|\bigcup_{j \in \{1,\ldots,|\mathcal{A}|\} \setminus \{i\}} CompStates(a_i, a_j)| \geq 5$. By looking at Fig. 2, we observe that, in our case, agreement checks for four actions must be done; for each selected action, an agreement check for each variable of the state is done.

5.2.3 Possible insights obtained from the results. Fig. 3 shows an example of the obtained results.[4] Specifically, in the middle columns of the first two tables, it reports the results of disagreement checks between agents $\mathtt{AG}_Y$ and $\mathtt{AG}_X$ when $\mathtt{AG}_Y$ plays actions a_4 (first table) and a_3 (second table); for each variable x of the state (each row), it reports the statistical difference of the variable w.r.t. the cases in which the two agents play the same action: the color of the cell identifies the $\hat{A}_{12}$ value, where red identifies values greater than 0.5 (i.e., x is greater when they disagree), and blue values lower than 0.5 (i.e., x is lower when they disagree).

The explanation of the different actions can be done by considering the disagreement checks for all the variables of the state (i.e., a whole column). By observing the results, we identify these categories of state differences that can explain the different actions of the agents:

- *Multiple variables cause.* This category refers to the situations in which several variables are significantly different from the agreement case. This is a common situation, as it is reasonable that several changes to the states in which the two agents agree, can introduce some factors on which the agents do not agree. For example, agent $\mathtt{AG}_X$ plays action a_6 instead of a_3 (second table), when the values of several variables are different from the agreement case (in which both agents play a_3). These variables show a situation in which the player is low of energy and lacks some items. By playing a_6, $\mathtt{AG}_X$ decides to recover energy, while, by playing a_3, $\mathtt{AG}_Y$ decides to continue the activity to collect items. These are two different game strategies.
- *Few variables cause.* In this category, just the variation of a variable (or few variables) makes the difference between the agreement and disagreement. This is the case when agent $\mathtt{AG}_Y$ plays action a_4 and agent $\mathtt{AG}_X$ plays action a_2 (first table); in this case, just two variables are significantly larger: x_1 indicates the period of the game and x_8 the availability of a particular resource. This means that the decision of agent $\mathtt{AG}_X$ is particularly influenced by the fact that later in the game a particular resource is available.

Further observations can be done regarding the variables:

- some variables are always involved in the explanation of the disagreement, like x_1, x_6, and x_8 in the second table in Fig. 3. It seems that these variables are important factors for the game, as different variations of their values lead to different decisions. In the game, x_6 and x_8 are related to the resources available to the player: it is reasonable that they are fundamental decision variables, as what is available to the character can greatly influence what it can do.
- some other variables like x_4 and x_5, have low influence on the disagreement. In the game, these variables record some characteristics of the character that do not have a high influence on the decision of which action to play.

In Fig. 3, the last column reports the results for the agreement case. The insights that can be obtained are similar (but dual) to those that can be obtained for the disagreement case.

6 THREATS TO VALIDITY

We discuss threats that may affect the validity of our approach [22].

External validity. The proposed approach has been applied to only one game provided by our industry partner and by only considering two reinforcement learning agents and one rule-based agent. It is not clear whether the approach is generalizable to other games and/or other types of agents. First of all, we agree with Briand et al. [3] that research conducted in collaboration with industry is necessarily "context-driven" as it must solve the concrete problem of the industry. In this aspect, our approach showed to be useful for our industry partner. In any case, as future work, we plan to apply/adapt the approach to other games having similar characteristics of the game targeted in this work.

[4] The actions for which there is no significant difference are not shown in the figure.

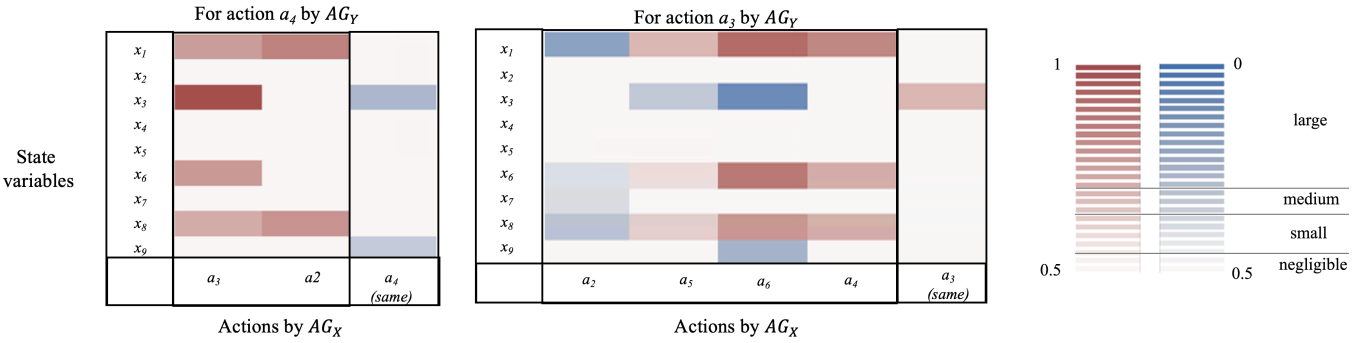

Figure 3: State analysis

Another threat to external validity is that the approach could not scale to games whose state is defined by several variables. For such games, abstraction of the game state should be employed.

Internal validity. The only assumption in the proposed method is that it is possible to collect actions for each specific given state in the test alignment phase (see Section 4.2). This is usually possible for automated agents, as such an architecture to allow for repeating state-action cycles is common as "gyms" for reinforcement learning.

A possible limitation threat to the application of the approach would be the involvement of human agents to explain their behaviour. It is unrealistic to expect human players to repeat hundreds of cycles for deciding actions for each of given arbitrary states. When we compare an automated agent and a human agent, we can tailor our method into an asymmetric way. An automated agent plays with the states the human agent encountered with test plays and only test pairs with these states are analyzed. This way excludes states that were encountered by the automated agent but not by the human agent, which we believe not critical.

The analysis step for comparing and analyzing the difference (see Section 5) is applicable only if there is a sufficient number of test pairs sharing the same set of states. However, it is difficult to naturally accumulate such test pairs and we believe our way of test alignment (see Section 4.2) is efficient and matches with the recent active use of automated agents.

Construct validity. In the state analysis (see Section 5.2), we try to understand which variables of the state are responsible for the disagreement of two agents w.r.t. to some actions. The presence of confounding factors could falsely identify some variable as responsible for the disagreement, when it is not. For example, in different experimental results, we observed the influence of the stage of the game on the disagreement between agents. However, we can easily imagine that the stage of the game is also correlated with other variables such as the energy of the character or the resources available, which may be the only true causes of disagreement. As future work, we plan to apply techniques to handle confounding factors.

7 RELATED WORK

In this section, we review works that, from different point of views, are related with our work.

Explanation techniques. In our work, we aim to understand the behaviours of game agents, specifically based on Deep Reinforcement Learning (DRL). As claimed by Burkart et al. [4], in Artificial Intelligence (AI), a detailed understanding of the model and its outputs is essential. Various studies focusing on the explainability of AI techniques, especially for DNN models, exist. For example, Zeiler and Fergus [23] introduced a visualization technique to observe the status of the intermediate feature layers inside the Convolutional Neural Network (CNN) during the classification. Goyal et al. [7] developed a technique to produce counterfactual visual explanations.

Differential testing. Our approach is inspired by differential testing [6], a testing technique which executes different versions of a system under test using the same test cases and then compares the outputs results [8]. Differential testing has been applied in various domains. For example, Misse-Chanabier et al. [11] proposed an approach to find bugs in Virtual Machine (VM) generators by comparing the difference of various VM versions. Prochnow and Yang [15] developed a tool, called *Diff Watch*, that automatically monitors the changes of deep learning libraries. Our approach distinguishes from the classical application of differential testing, as it compares different agents and not different versions of the agent.

Testing reinforcement learning. The agents that we consider in our work are mainly based on reinforcement learning. Some approaches have been proposed to test reinforcement learning. Zolfagharian et al. [25] proposed a search-based testing approach to test DRL agents, aiming to find the failing executions. Tappler et al. [19] also use SBT to evaluate the safety and performance of deep RL agents. Nikanjam et al. [12] proposed an approach to detect bugs for various DRL applications such as OpenAI Gym and Dopamine. Trujillo et al. [20] investigate different coverage criteria for testing DRL applications and find that neuron coverage is not sufficient.

Game testing. The agents whose behaviour we assess in our work, are ultimately used for testing a GaaS game. Testing games is a wide research area. However, existing game testing still needs significant advances [14]. Paduraru et al. [13] developed a tool, called *RiverGame*, to detect the available operations and states such that the game under test can be played and evaluated automatically. Zhen et al. [24] proposed an approach, called *Wuji*, that leverages evolutionary algorithms, DRL and multi-objective optimization to perform automatic game testing. Similarly, Ariyurek et al. [1] designed an approach to test games combing both synthetic agents and human-like agents, in which the human-like agent aims to

achieve high scores and, meanwhile, the synthetic agents aim to explore the unintended game transitions using Monte Carlo tree search (MCTS). Shirzadehhajimahmood et al. [18] proposed an agent-based approach to produce robust automated tests 2D/3D virtual games with the help of path finding techniques.

8 CONCLUDING REMARKS

Game agents are widely used to test a game before release, in particular when the release cycles are frequent. In this context, a fundamental aspect is to understand the behaviour of an agent. In this work, we proposed an approach to compare and explain the behaviour of two agents, that identifies when two agents agree and disagree in applying specific actions, and which are the state features that lead to agreement and disagreement.

Our method assumes that the agents have deterministic behaviour, as these are the type of agents that we were provided. However, we could extend the approach to support non-deterministic agents that can apply different actions from the same state. In this case, we would need to repeat the agent execution multiple times from the same state, to get an understanding of the distribution of the possible actions; we leave this as future work.

As future work, we plan to extend the technique by associating the results on disagreement with the performance (e.g., score, time, money spent) obtained by the actions chosen by the two agents (either of a single game step or along multiple steps). In this way, we could try to test the game balance, by checking whether there are specific agents (representing possible game players) that could break it (e.g., winning too easily, or loosing motivation in playing).

ACKNOWLEDGMENTS

X. Zhang, P. Arcaini, and F. Ishikawa are supported by ERATO HASUO Metamathematics for Systems Design Project (No. JPMJER1603), JST, Funding Reference number: 10.13039/501100009024 ERATO; and by Engineerable AI Techniques for Practical Applications of High-Quality Machine Learning-based Systems Project (Grant Number JPMJMI20B8), JST-Mirai. T. Takisaka is supported by Research Fund for International Young Scientists (No. 62150410437), NSFC.

REFERENCES

[1] Sinan Ariyurek, Aysu Betin-Can, and Elif Surer. 2021. Automated Video Game Testing Using Synthetic and Humanlike Agents. *IEEE Transactions on Games* 13, 1 (2021), 50–67. https://doi.org/10.1109/TG.2019.2947597

[2] Joakim Bergdahl, Camilo Gordillo, Konrad Tollmar, and Linus Gisslén. 2020. Augmenting Automated Game Testing with Deep Reinforcement Learning. In *2020 IEEE Conference on Games (CoG)*. IEEE, 600–603. https://doi.org/10.1109/CoG47356.2020.9231552

[3] Lionel C. Briand, Domenico Bianculli, Shiva Nejati, Fabrizio Pastore, and Mehrdad Sabetzadeh. 2017. The Case for Context-Driven Software Engineering Research: Generalizability Is Overrated. *IEEE Software* 34, 5 (2017), 72–75. https://doi.org/10.1109/MS.2017.3571562

[4] Nadia Burkart and Marco F. Huber. 2021. A Survey on the Explainability of Supervised Machine Learning. *J. Artif. Int. Res.* 70 (may 2021), 245–317. https://doi.org/10.1613/jair.1.12228

[5] Youri Coppens, Kyriakos Efthymiadis, Tom Lenaerts, and Ann Nowe. 2019. Distilling Deep Reinforcement Learning Policies in Soft Decision Trees. In *IJCAI 2019 Workshop on Explainable Artificial Intelligence*. 1–6.

[6] Robert B. Evans and Alberto Savoia. 2007. Differential Testing: A New Approach to Change Detection. In *Proceedings of the the 6th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on The Foundations of Software Engineering* (Dubrovnik, Croatia) *(ESEC-FSE '07)*. Association for Computing Machinery, New York, NY, USA, 549–552. https://doi.org/10.1145/1287624.1287707

[7] Yash Goyal, Ziyan Wu, Jan Ernst, Dhruv Batra, Devi Parikh, and Stefan Lee. 2019. Counterfactual visual explanations. In *International Conference on Machine Learning*. PMLR, 2376–2384.

[8] Muhammad Ali Gulzar, Yongkang Zhu, and Xiaofeng Han. 2019. Perception and Practices of Differential Testing. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 71–80. https://doi.org/10.1109/ICSE-SEIP.2019.00016

[9] Daniel Hein, Steffen Udluft, and Thomas A. Runkler. 2019. Generating Interpretable Reinforcement Learning Policies Using Genetic Programming. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion* (Prague, Czech Republic) *(GECCO '19)*. Association for Computing Machinery, New York, NY, USA, 23–24. https://doi.org/10.1145/3319619.3326755

[10] Barbara Kitchenham, Lech Madeyski, David Budgen, Jacky Keung, Pearl Brereton, Stuart Charters, Shirley Gibbs, and Amnart Pohthong. 2017. Robust Statistical Methods for Empirical Software Engineering. *Empirical Softw. Engg.* 22, 2 (apr 2017), 579–630. https://doi.org/10.1007/s10664-016-9437-5

[11] Pierre Misse-chanabier, Guillermo Polito, Noury Bouraqadi, Stéphane Ducasse, Luc Fabresse, and Pablo Tesone. 2022. Differential Testing of Simulation-Based Virtual Machine Generators. In *Reuse and Software Quality*. Springer International Publishing, Cham, 103–119.

[12] Amin Nikanjam, Mohammad Mehdi Morovati, Foutse Khomh, and Houssem Ben Braiek. 2022. Faults in deep reinforcement learning programs: a taxonomy and a detection approach. *Automated Software Engineering* 29, 1 (2022), 1–32.

[13] Ciprian Paduraru, Miruna Paduraru, and Alin Stefanescu. 2022. RiverGame - a game testing tool using artificial intelligence. In *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*. 422–432. https://doi.org/10.1109/ICST53961.2022.00048

[14] Cristiano Politowski, Fabio Petrillo, and Yann-Gaël Guéhéneuc. 2021. A Survey of Video Game Testing. In *2021 IEEE/ACM International Conference on Automation of Software Test (AST)*. 90–99. https://doi.org/10.1109/AST52587.2021.00018

[15] Alexander Prochnow and Jinqiu Yang. 2022. DiffWatch: Watch Out for the Evolving Differential Testing in Deep Learning Libraries. In *2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 46–50. https://doi.org/10.1145/3510454.3516835

[16] Erika Puiutta and Eric M. S. P. Veith. 2020. *Machine Learning and Knowledge Extraction*. Springer International Publishing, Chapter Explainable Reinforcement Learning: A Survey, 77–95.

[17] Ronnie E. S. Santos, Cleyton V. C. Magalhães, Luiz Fernando Capretz, Jorge S. Correia-Neto, Fabio Q. B. da Silva, and Abdelrahman Saher. 2018. Computer Games Are Serious Business and so is Their Quality: Particularities of Software Testing in Game Development from the Perspective of Practitioners. In *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement* (Oulu, Finland) *(ESEM '18)*. Association for Computing Machinery, New York, NY, USA, Article 33, 10 pages. https://doi.org/10.1145/3239235.3268923

[18] Samira Shirzadehhajimahmood, I. S. W. B. Prasetya, Frank Dignum, Mehdi Dastani, and Gabriele Keller. 2021. Using an Agent-Based Approach for Robust Automated Testing of Computer Games. In *Proceedings of the 12th International Workshop on Automating TEST Case Design, Selection, and Evaluation* (Athens, Greece) *(A-TEST 2021)*. Association for Computing Machinery, New York, NY, USA, 1–8. https://doi.org/10.1145/3472672.3473952

[19] Martin Tappler, Filip Cano Córdoba, Bernhard K. Aichernig, and Bettina Könighofer. 2022. Search-Based Testing of Reinforcement Learning. *CoRR* abs/2205.04887 (2022). https://doi.org/10.48550/arXiv.2205.04887 arXiv:2205.04887

[20] Miller Trujillo, Mario Linares-Vásquez, Camilo Escobar-Velásquez, Ivana Dusparic, and Nicolás Cardozo. 2020. Does Neuron Coverage Matter for Deep Reinforcement Learning? A Preliminary Study. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops* (Seoul, Republic of Korea) *(ICSEW'20)*. Association for Computing Machinery, New York, NY, USA, 215–220. https://doi.org/10.1145/3387940.3391462

[21] Ulf Wilhelmsson, Wei Wang, Ran Zhang, and Marcus Toftedahl. 2022. Shift from game-as-a-product to game-as-a-service research trends. *Service Oriented Computing and Applications* 16 (2022), 79–81.

[22] Claes Wohlin, Per Runeson, Martin Hst, Magnus C. Ohlsson, Bjrn Regnell, and Anders Wessln. 2012. *Experimentation in Software Engineering*. Springer Publishing Company, Incorporated.

[23] Matthew D Zeiler and Rob Fergus. 2014. Visualizing and understanding convolutional networks. In *European conference on computer vision*. Springer, 818–833.

[24] Yan Zheng, Xiaofei Xie, Ting Su, Lei Ma, Jianye Hao, Zhaopeng Meng, Yang Liu, Ruimin Shen, Yingfeng Chen, and Changjie Fan. 2019. Wuji: Automatic Online Combat Game Testing Using Evolutionary Deep Reinforcement Learning. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 772–784. https://doi.org/10.1109/ASE.2019.00077

[25] Amirhossein Zolfagharian, Manel Abdellatif, Lionel Briand, Mojtaba Bagherzadeh, et al. 2022. Search-Based Testing Approach for Deep Reinforcement Learning Agents. *arXiv preprint arXiv:2206.07813* (2022).